TAKE-HOME ASSESSMENT

# Software Engineer, AI & Product Engineering

Datastraw Technologies | About 4-5 hours of work | Submit within 7 days of receiving this document

## Overview and ground rules

This assessment asks you to build a small slice of the kind of product you would work on at Datastraw. It is a real engineering task, kept deliberately small, so that it respects your time.

- **Time:** plan for about 4-5 hours, including deployment. Please do not spend much more. We would rather see good judgment and a short list of what you cut than a polished app that took a weekend.
- **Deadline:** 7 days from the day you receive this document.
- **AI tools are allowed.** Use Copilot, Claude, ChatGPT or anything else you normally would. We care about what you build, the decisions you make and whether you understand your own code.
- **Work alone.** The code and the write-up should be yours.
- **Cost:** a free tier of any LLM provider is enough. If getting an API key is a problem for you, tell us and we will help.

## The scenario

Verdant & Co is a fictional online skincare brand. Customers message the brand on a chat app (think WhatsApp) with questions such as “Where is my order?” or “Can I return this?”. A small support team answers them, and they want a tool that helps them answer faster.

You are building the first version of that tool: a small web app with a backend. It does three things:

1. **Receives customer messages.** The chat system sends every new customer message to your server as an HTTP request (a “webhook”). Your server saves it.
2. **Drafts a reply with AI.** For each new message, your server asks an LLM to write a suggested reply, using the brand’s policies in the “Knowledge base” section at the end.
3. **Lets a support agent review and approve.** An agent signs in to your web app, reads the conversation and the AI’s draft, edits the draft if needed, and approves it.

### An example, start to finish

- Aisha, a customer, messages: “Hi, my order #4471 hasn’t arrived yet. Can I change the delivery address?”
- The message reaches your server and is saved in Aisha’s conversation.
- Your server asks the LLM for a draft reply based on the brand’s policies, and saves it.
- A support agent signs in. Aisha’s conversation is at the top of the list. The agent reads the message and the draft, fixes a sentence, and clicks Approve.
- The approved reply is saved in Aisha’s conversation. You do not send anything to a real chat app; saving it is enough.

Treat it as something a small team would run in production next week, not as a throwaway demo.

Because there is no real chat app, your web app also includes a simple **Customer view**. A switch at the top lets us flip between **Customer** and **Agent**, so we can play both sides of the conversation and test the AI replies ourselves.

## What to build

### 1. Receive and store messages

- Create an endpoint (for example `POST /webhook`) that accepts customer messages in the format shown in the "Webhook format" section at the end.
- Save them in PostgreSQL or a similar relational database. Keep track of the customer (id, name, phone), the conversation, and each message (text, time, and whether it came from the customer or from an agent).
- Each customer has one open conversation. A new message from a customer who already has one is added to it.

### 2. Draft a reply with AI

- When a customer message arrives, generate a suggested reply with an LLM, using the knowledge base as the source for policies.
- Write replies an agent could send with little editing: short, polite and accurate.
- Save each draft with the message it answers, so the agent sees it when they open the conversation.
- A draft is only a suggestion. It is never sent to the customer automatically.

### 3. Agent web app

- **Sign in:** only signed-in agents can see all conversations. Any simple, sensible approach is fine.
- **Conversation list:** each customer's name, a preview of their latest message and the time, with the newest activity first.
- **Conversation page:** all messages in order, with the AI's draft in an editable box and an Approve button.
- **Approve:** saves the (possibly edited) text as a reply in the conversation and marks the draft as used.

### 4. Customer and agent toggle

- A switch at the top of the app flips between the **Customer** view and the **Agent** view. It is a test tool, so we can play both sides of the conversation ourselves.
- **Customer view:** choose one of a few test customers (or type a name and phone number), then type and send a message. The customer sees their own conversation, including the agent's approved replies. They never see drafts.
- A message sent from the Customer view must be handled exactly like a message delivered to your webhook: same saving, same AI draft.
- The Customer view does not need a sign-in. The Agent view does.

**You are done when**

- we can open the Customer view, send one of the sample messages, switch to the Agent view, sign in, and see the message there;
- the conversation shows an AI-drafted reply;
- we can edit the draft, approve it, switch back to the Customer view, and see the approved reply there;
- we can also send a test request straight to your webhook endpoint and get the same result.

**Not required:** a polished design, real WhatsApp integration, file or image messages, or multi-language support.

## What is open to you

We do not prescribe the stack. Choose what lets you do your best work in the time, and tell us why in your README.

- **Backend and database:** any language and framework. We use Supabase and PostgreSQL day to day, so those are a good fit, but they are not required.
- **Frontend:** React is preferred.
- **LLM provider:** any, including OpenRouter, DeepInfra, OpenAI, Anthropic or a free tier.
- **Retrieval:** the knowledge base is small. Keyword search, embeddings, a vector database, or putting all of it in the prompt are all acceptable. Explain your choice and when it would stop working.
- **Deployment:** your app must be deployed and working, so we can test it ourselves. Free tiers on Vercel, Netlify, Railway, Render, Supabase or similar are fine. We must also be able to run it locally from your README.

## What to submit

After completing the assessment, **please submit the following four items through the Google Form we share with you:**

1. **A deployed link** to your working app, with test sign-in details for an agent account and the URL of your webhook endpoint. We will open the app and send our own test messages to the endpoint, so please keep it live until you hear from us.
2. **A GitHub repository** (public, or private shared with the account we give you). Do not commit secrets: keep API keys in environment variables and include an example env file.
3. **A short README** in the repo, one page at most, covering:
   - how to run the project locally;
   - the main decisions you made and what you left out because of the time limit;
   - where an AI tool helped, and one place it got something wrong;
   - your answers to the two questions below.

4. **A demo video of 3 minutes or less** (unlisted link is fine). Show it working end to end, then cover: what you built, one key technical decision, how the AI part works, and one thing you would improve.

Note: All required links, and the demo video must be submitted through the **Google Form: [Google Form Link - https://forms.gle/AdQUpTfUwJ2U3NVs8]**

## Two short questions

Answer these in your README, in two or three sentences each.

1. A customer message arrives at 2 a.m. and something in your system fails. What does the agent see the next morning, and how would you find out what went wrong?
2. This now has to serve 20 brands, each with its own knowledge base and agents. What would you change first, and why?

## How we review

We read every submission. We look at:

- whether it works end to end on your deployed link, and whether it keeps working when things go wrong;
- how you model and store the data;
- how the AI part behaves, including when the knowledge base does not have what the customer asked for;
- whether the code is clear and simple enough for someone else to take over;
- how clearly you explain your decisions in the README, your two answers and the video.

A good submission handles the situations a real system meets in production, not only the happy path. We would rather see fewer features done carefully.

We aim to reply to every submission within 48 hours. Shortlisted candidates will be invited to talk through their solution with us.

## Webhook format

Messages sent from your Customer view should be handled the same way as messages delivered to your webhook. We will also send test requests straight to your deployed endpoint, as a JSON `POST` shaped like the sample below. You can send them yourself with curl, Postman or a script. It is a simplified format made for this task, not the real WhatsApp one.

- Each request carries an `X-Signature: sha256=<hex>` header: the HMAC-SHA256 of the raw body, made with the test secret `verdant-demo-secret-2026`.
- Reply with a `2xx` status once you have handled the message.

```
{
  "event_id": "evt_7f3a91c2",
  "type": "message.received",
  "brand_id": "verdant-co",
  "timestamp": "2026-11-03T10:15:22Z",
  "customer": {
    "id": "cust_1042",
    "name": "Aisha Khan",
    "phone": "+919800000001"
  },
  "message": {
    "id": "msg_88213",
    "type": "text",
    "text": "Hi, my order #4471 hasn't arrived yet. Can I change the delivery address?"
  }
}
```

## Knowledge base

Use this as the only source of truth for the suggested replies.

| Topic | Policy |
|---|---|
| **Shipping** | Orders ship within 24 hours, Monday to Saturday. Delivery takes 3-5 business days in metro cities and 5-8 business days elsewhere in India. |
| **Address change** | The delivery address can be changed only before the order has shipped, and only if the customer contacts us within 12 hours of placing it. After shipping, the address cannot be changed. |
| **Cancellation** | An order can be cancelled before it ships. After it ships, the customer can refuse delivery and a refund is issued once the parcel returns to us. |
| **Returns** | Returns are accepted within 7 days of delivery for unopened, unused products. Opened products are accepted only if damaged or if the wrong item was sent. |
| **Refunds** | Approved refunds go back to the original payment method within 5-7 business days after we receive the returned parcel. |
| **Cash on delivery** | Cash on delivery is available for orders under ₹3,000, with a ₹50 fee. |
| **Ingredients** | All Verdant & Co products are fragrance-free and vegan. |

## Sample customer messages

Use these in the Customer view, or your own, to test your solution:

1. “Hi, my order #4471 hasn’t arrived yet. Can I change the delivery address?”
2. “What is your return policy?”
3. “Do you deliver to Dubai?”
4. “Is cash on delivery available for a ₹3,500 order?”
5. “I think I’m allergic to the serum I bought. What should I do?”

Please submit your completed assignment and all required files, links, and video through this Google Form